

动态规划

《算法设计与分析》

信息科学与技术学院 曾华琳



01



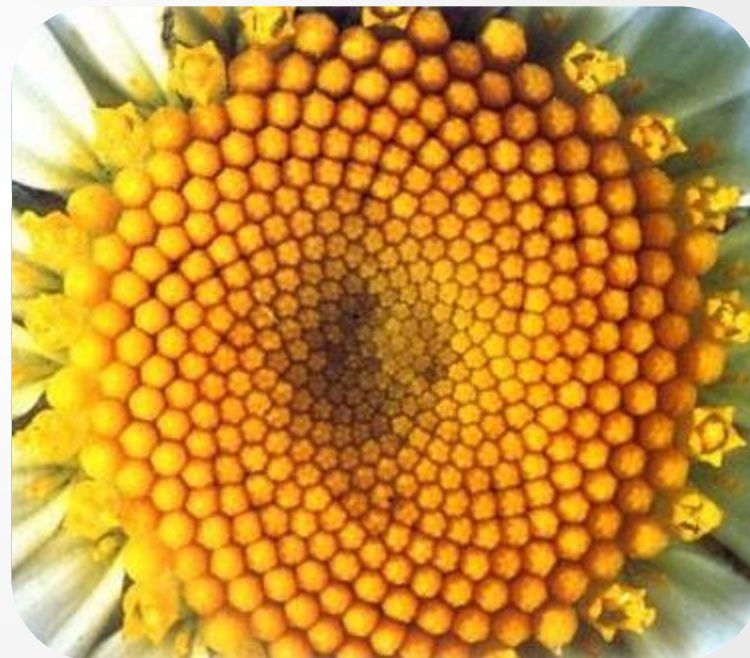
引言

History does not occur again



一、引言

自然界中的斐波那契





一、引言

■ 考虑Fibonacci序列 $F(n)$

$$F(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

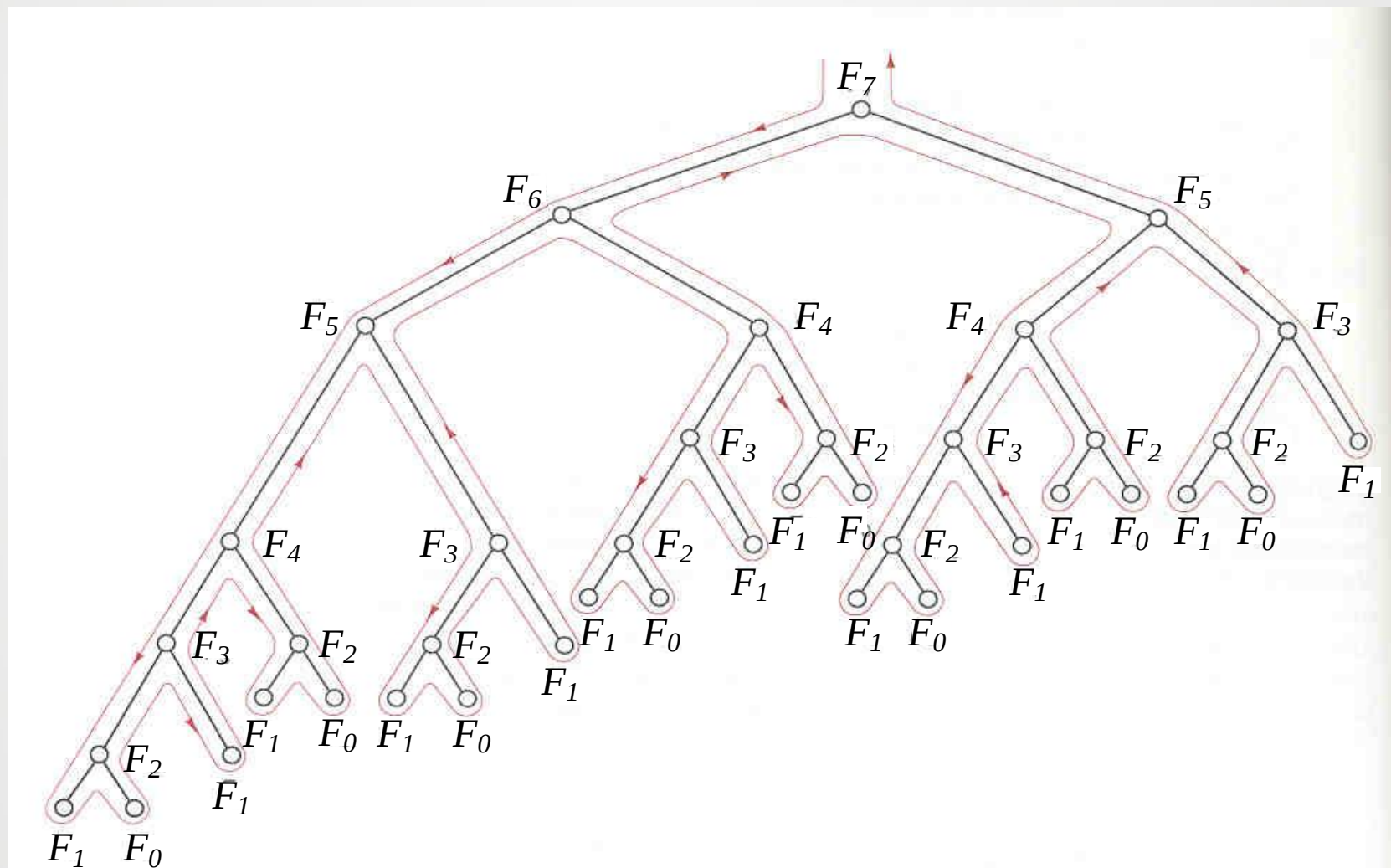
n	0	1	2	3	4	5	6	7	8	9	10
$F(n)$	1	1	2	3	5	8	13	21	34	55	89

递归算法的伪代码:

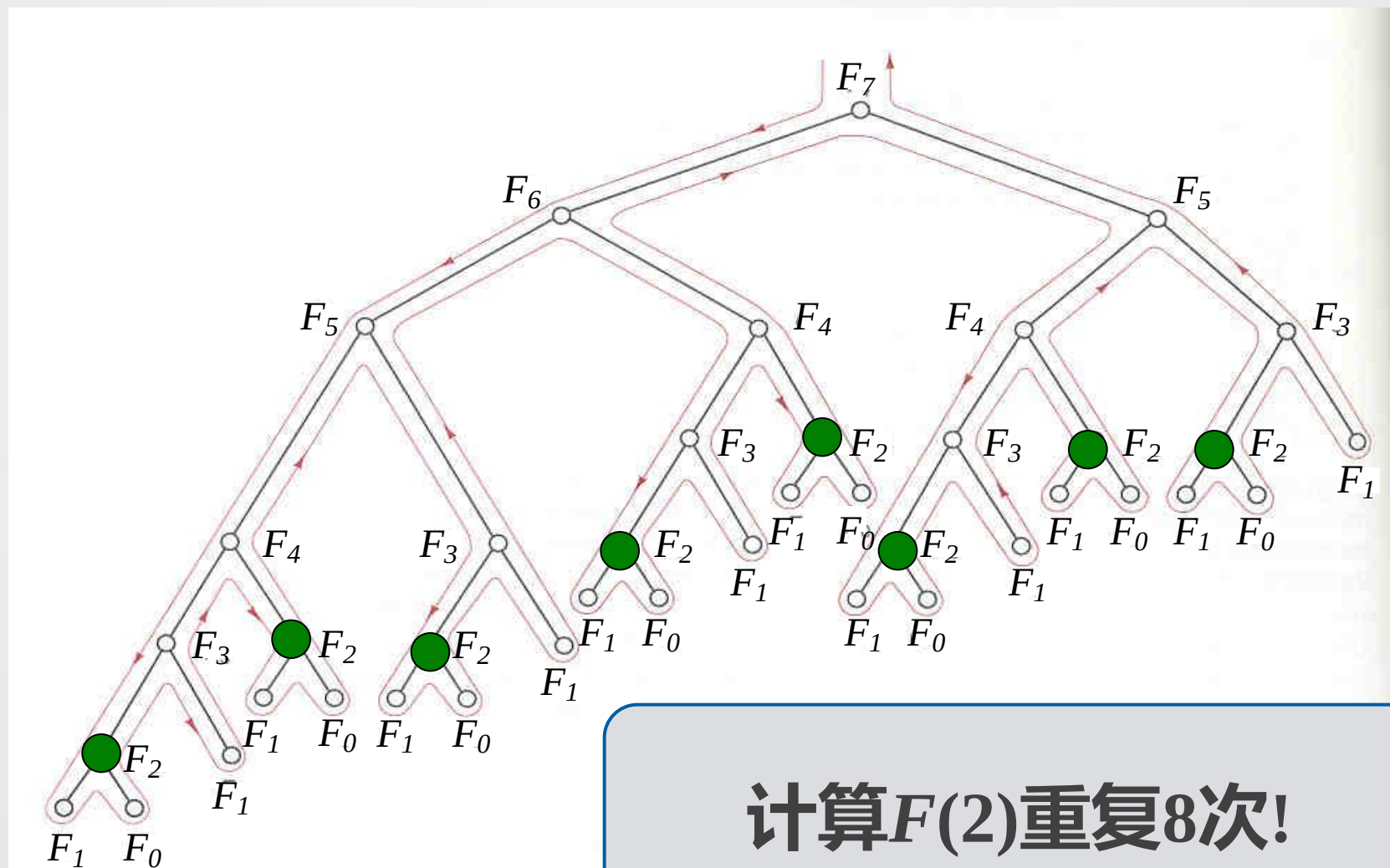
$F(n)$

```
1  if  $n=0$  or  $n=1$  then return 1
2  else return  $F(n-1) + F(n-2)$ 
```

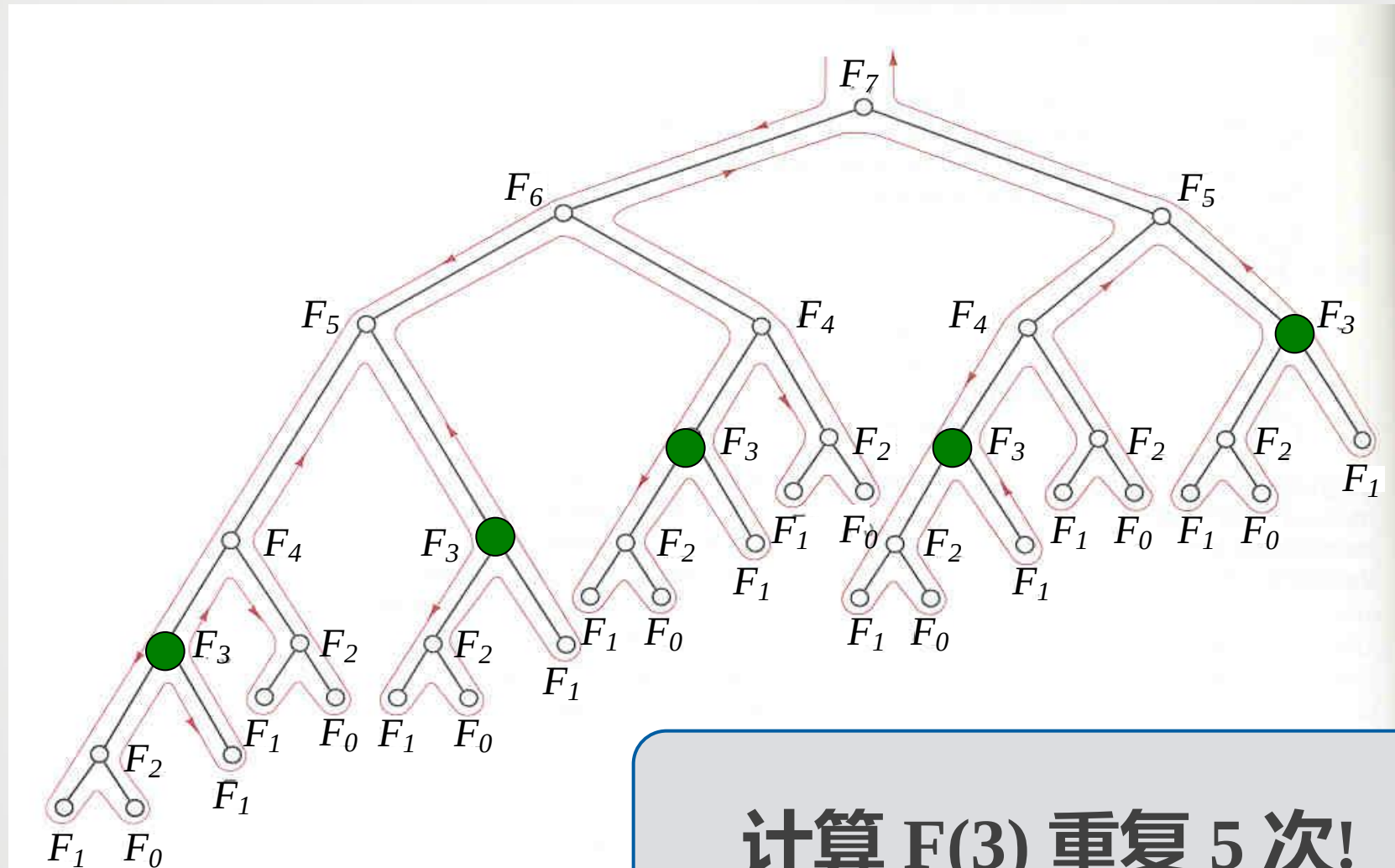
计算 $F(7)$



计算 $F(7)$



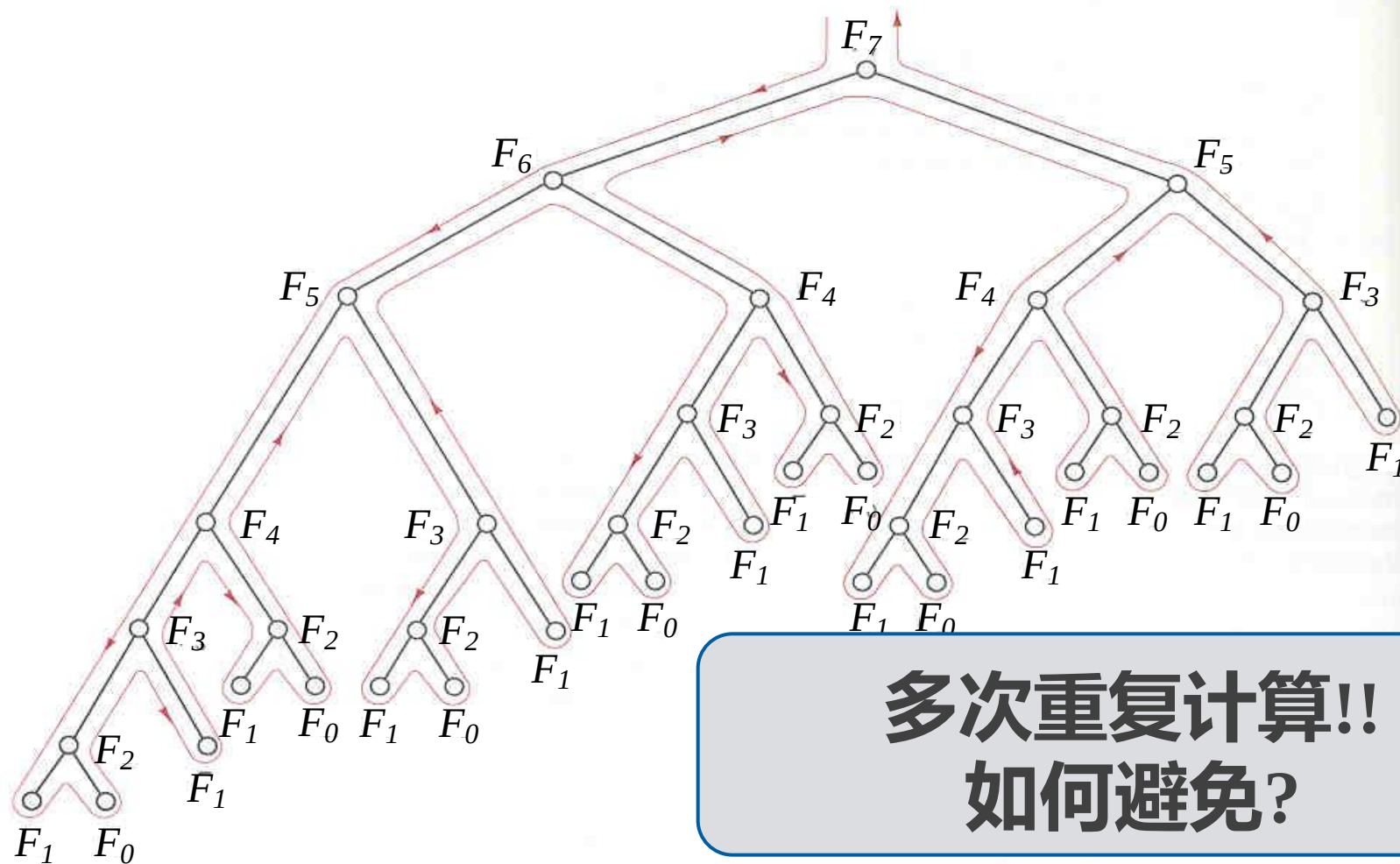
The execution of $F(7)$



计算 $F(3)$ 重复 5 次!

计算 $F(7)$

历史在重演
冗余在出现





改进的想法

■ 备忘录

当 $F1(i)$ 被计算后，保存它的值；

当再次计算 $F1(i)$ 时，只需要从内存中取出即可；

$F1(n)$

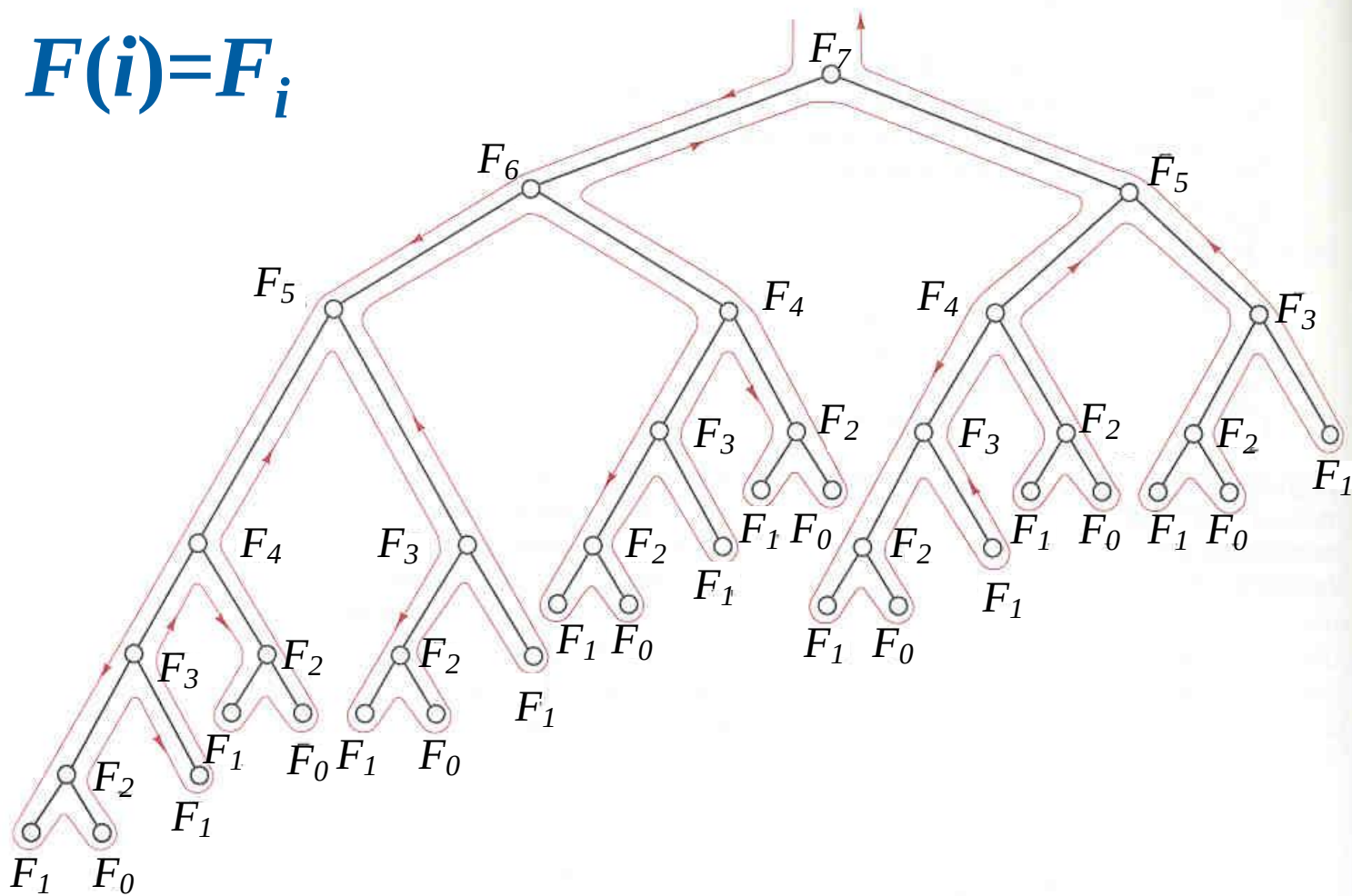
```
1 if  $v[n] < 0$  then  
2    $v[n] \leftarrow F1(n-1) + F1(n-2)$   
3 return  $v[n]$ 
```

Main()

```
1  $v[0] = v[1] \leftarrow 1$   
2 for  $i \leftarrow 2$  to  $n$  do  
3    $v[i] = -1$   
4 output  $F1(n)$ 
```

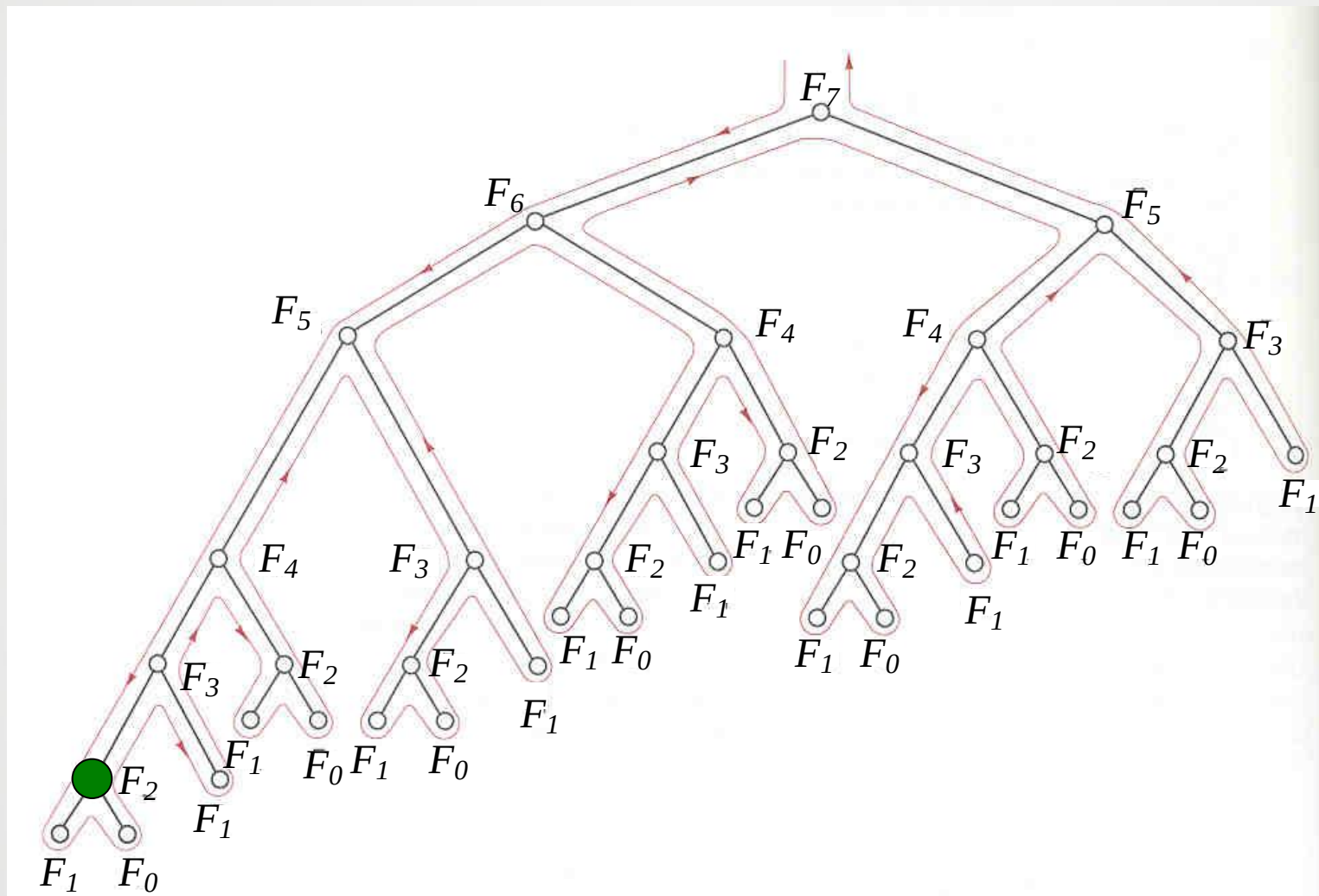
再计算 $F(7)$

$$F(i)=F_i$$



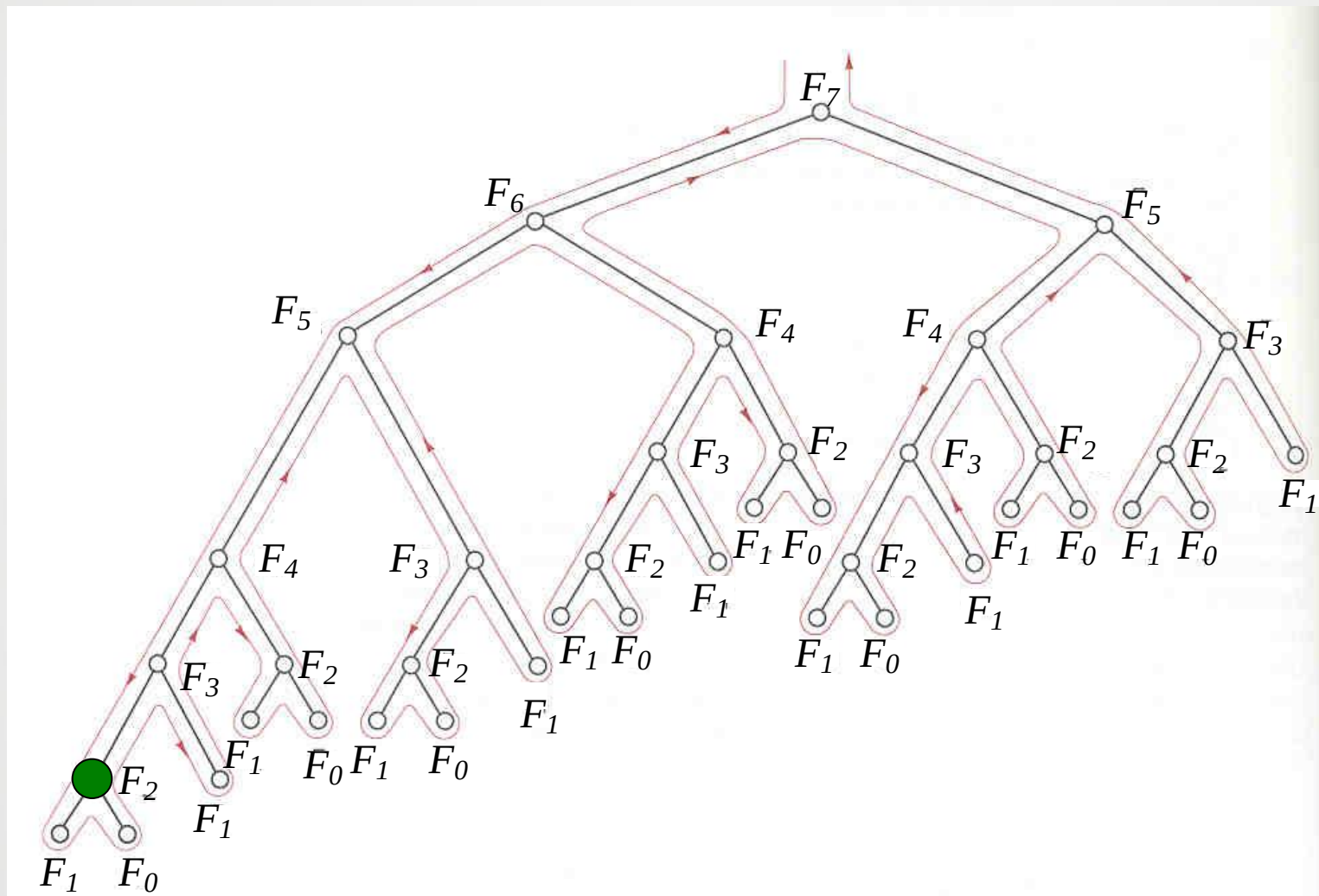
$v[0]$	1
$v[1]$	1
$v[2]$	-1
$v[3]$	-1
$v[4]$	-1
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

再计算 $F(7)$



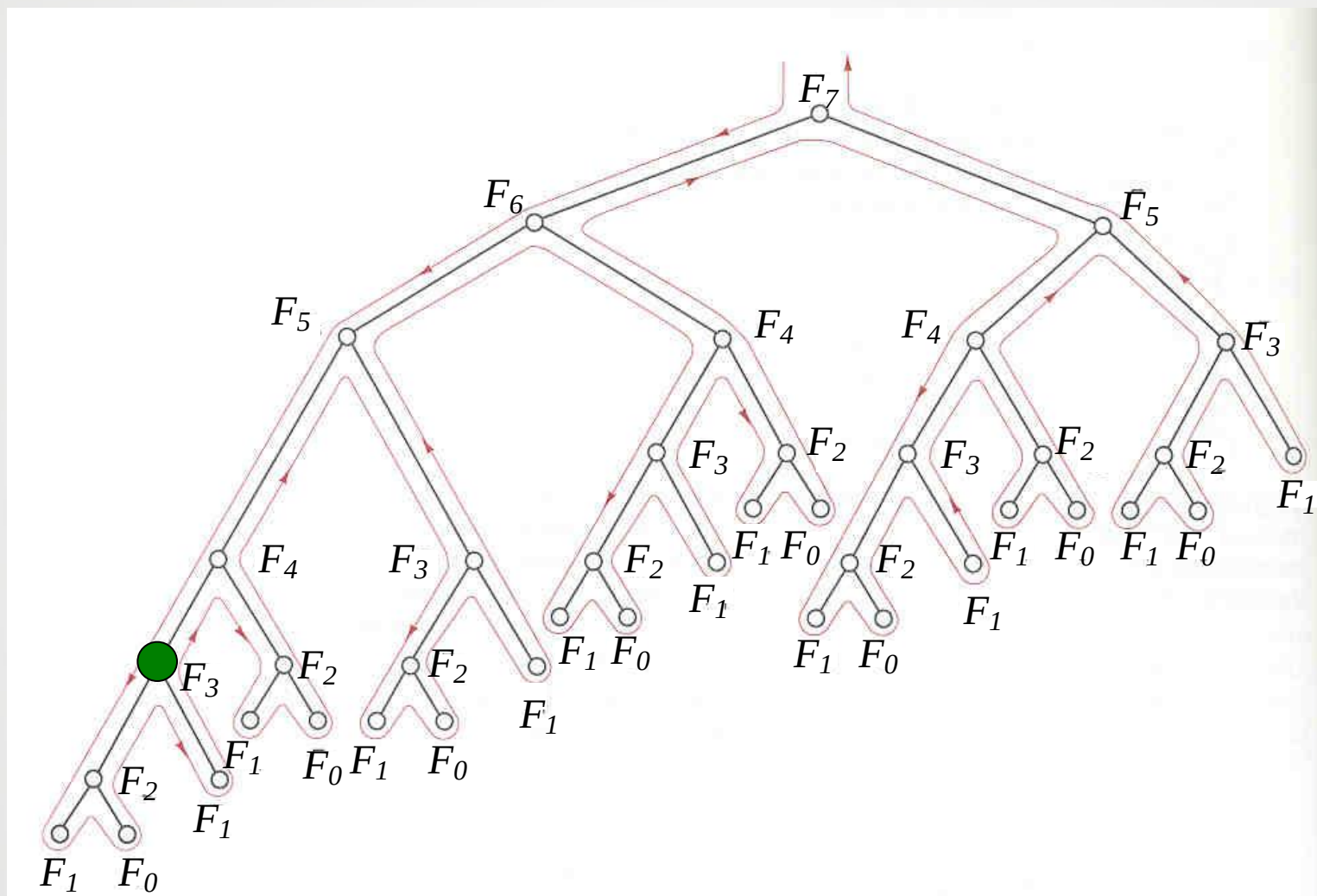
$v[0]$	1
$v[1]$	1
$v[2]$	-1
$v[3]$	-1
$v[4]$	-1
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

再计算 $F(7)$



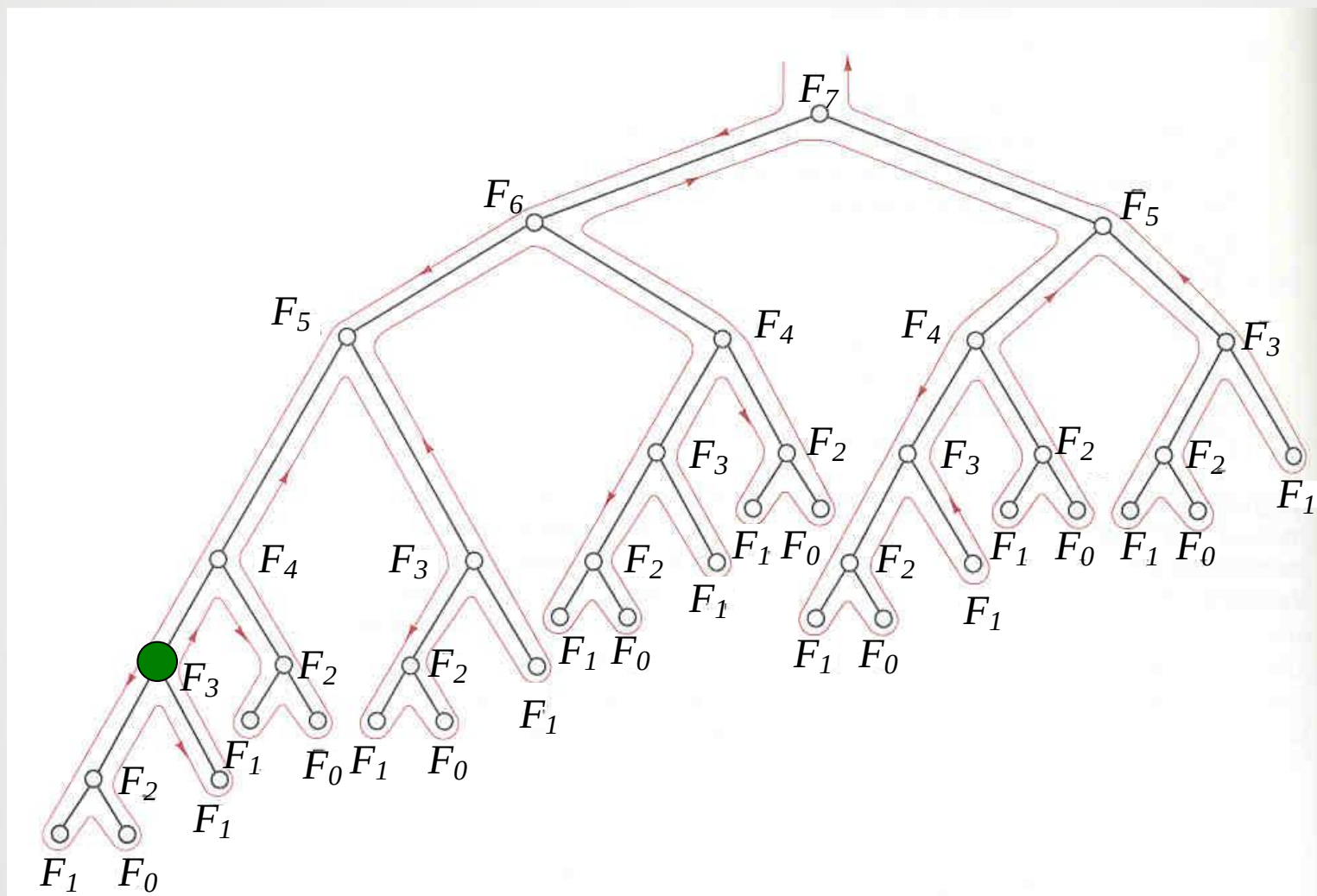
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	-1
$v[4]$	-1
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

再计算 $F(7)$



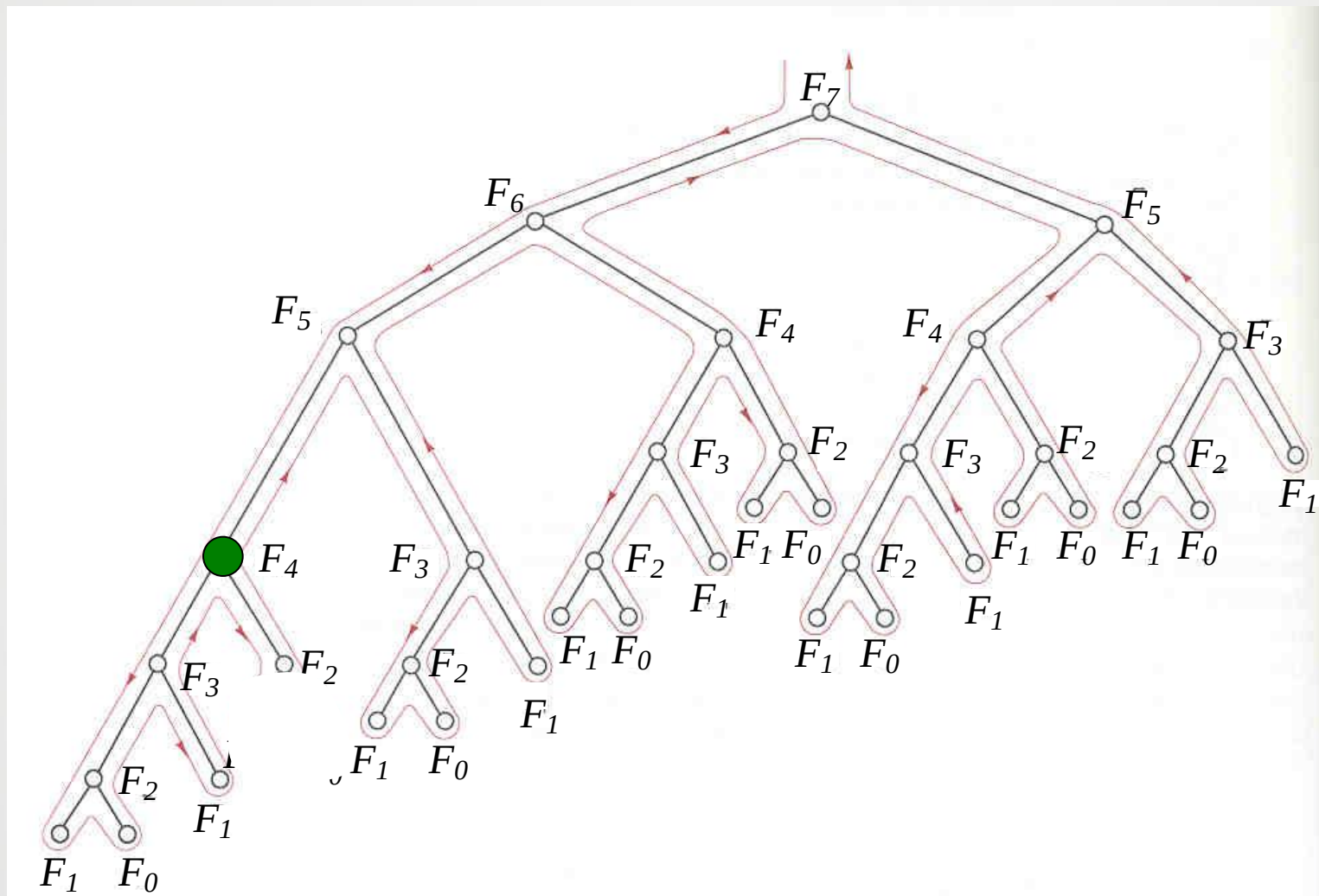
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	-1
$v[4]$	-1
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

再计算 $F(7)$



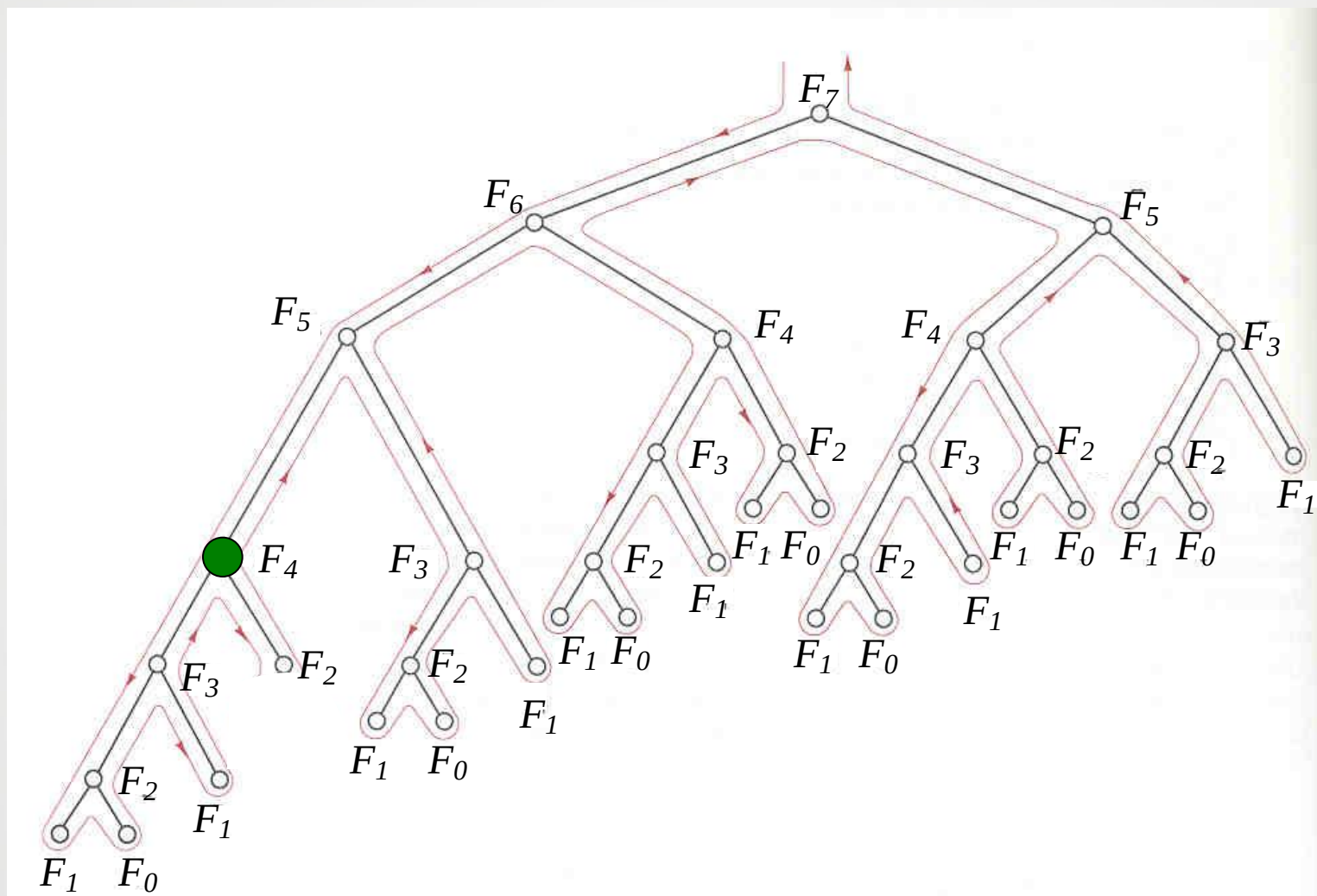
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	-1
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

再计算 $F(7)$



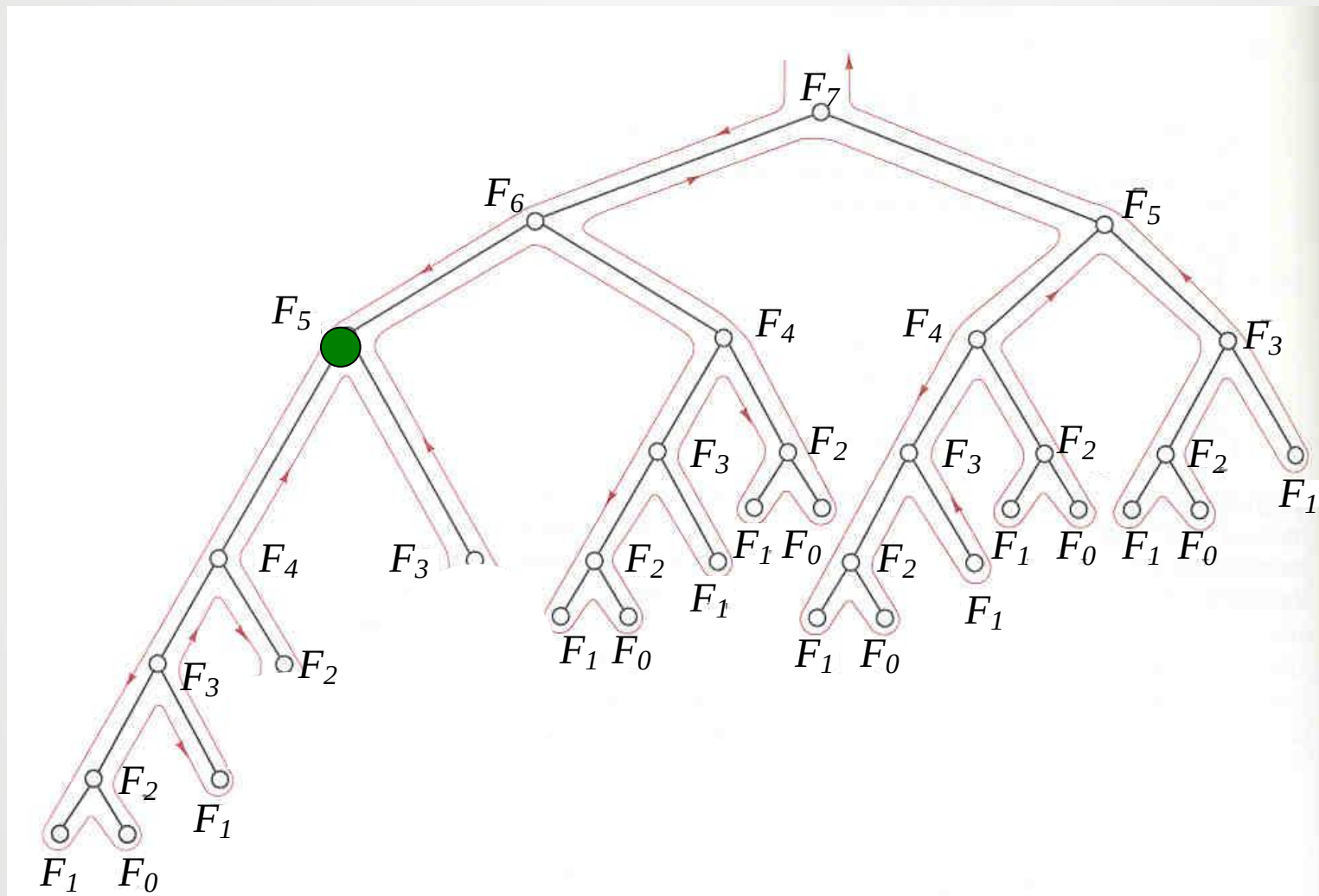
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	-1
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

再计算 $F(7)$



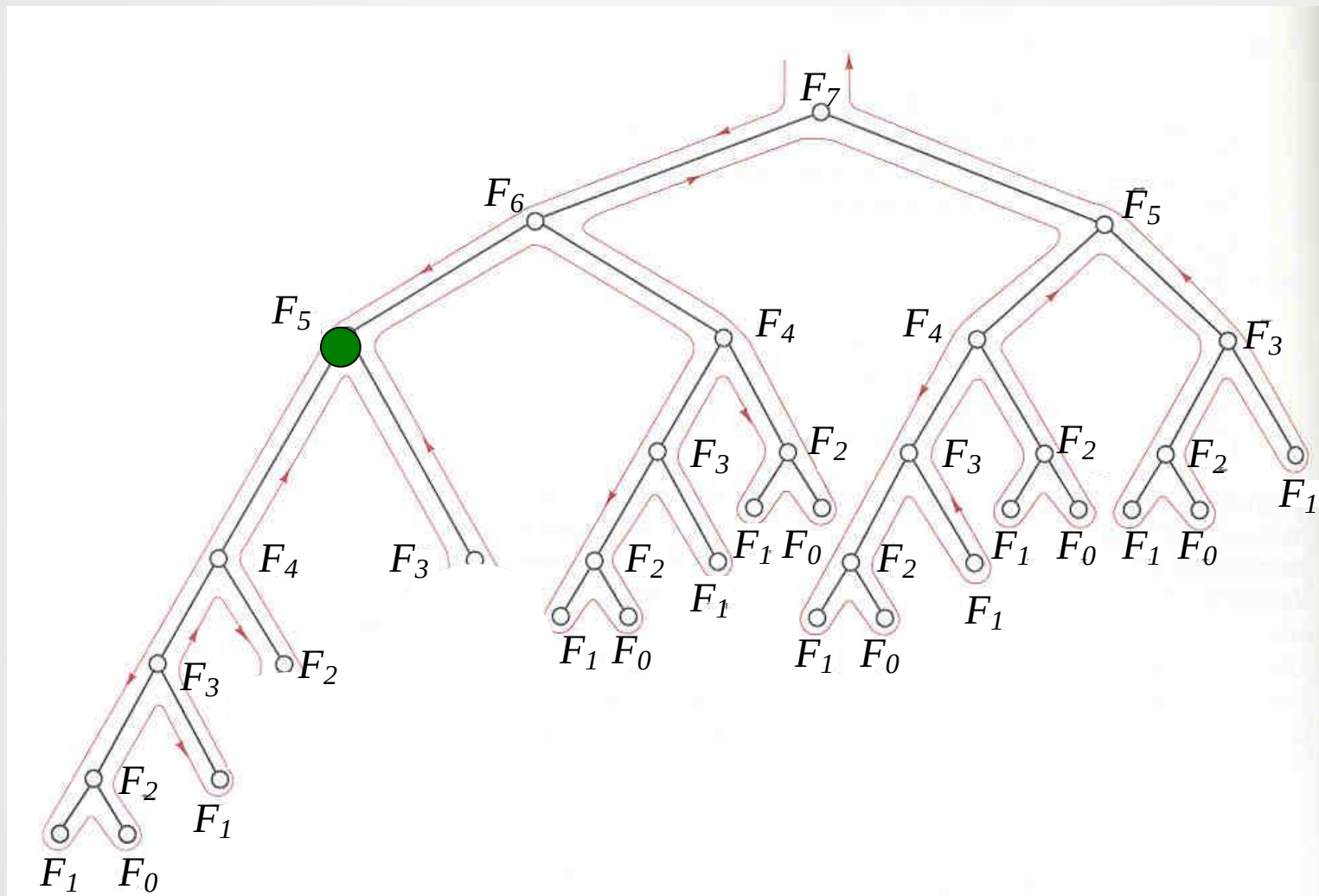
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	5
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

再计算 $F(7)$



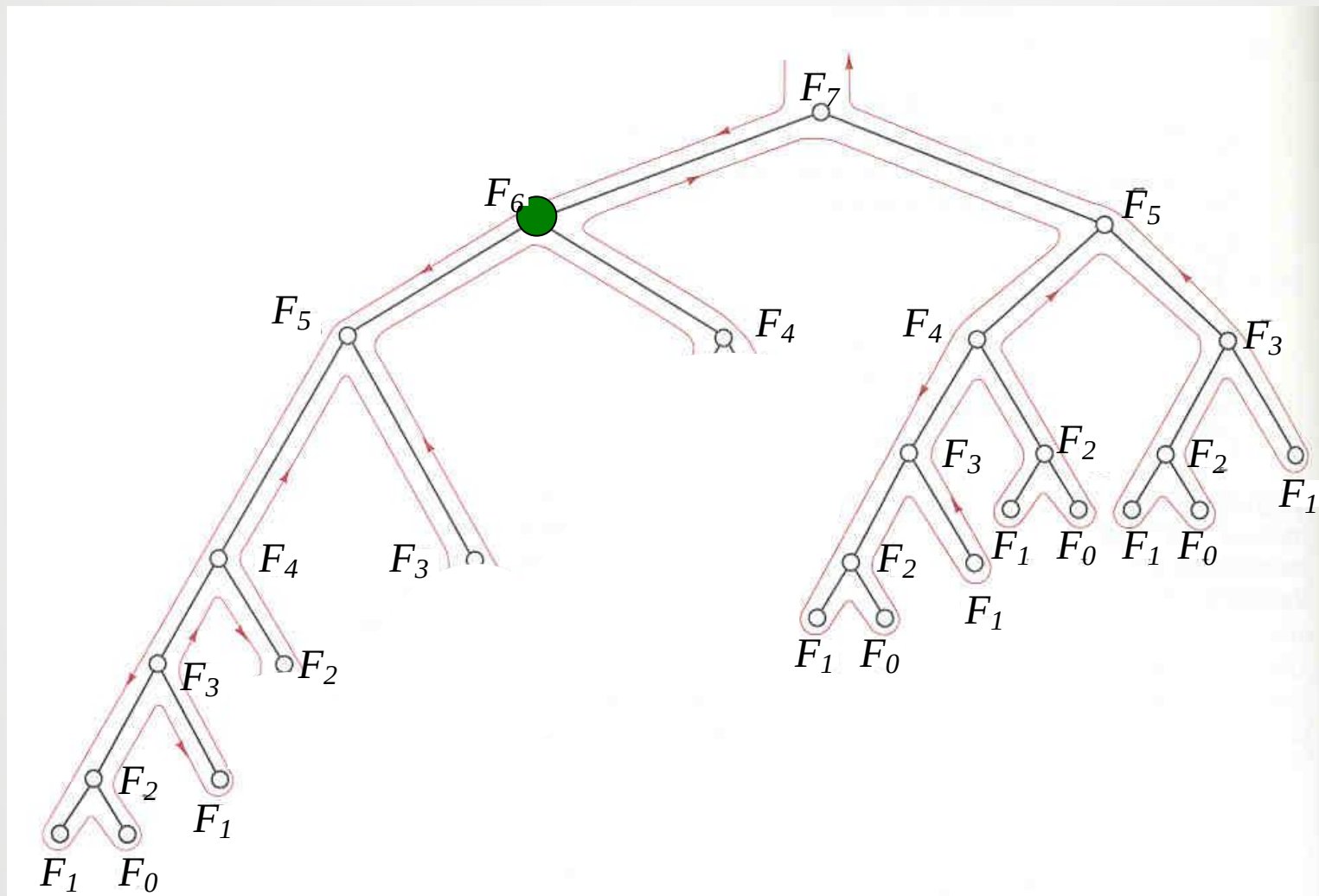
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	5
$v[5]$	-1
$v[6]$	-1
$v[7]$	-1

再计算 $F(7)$



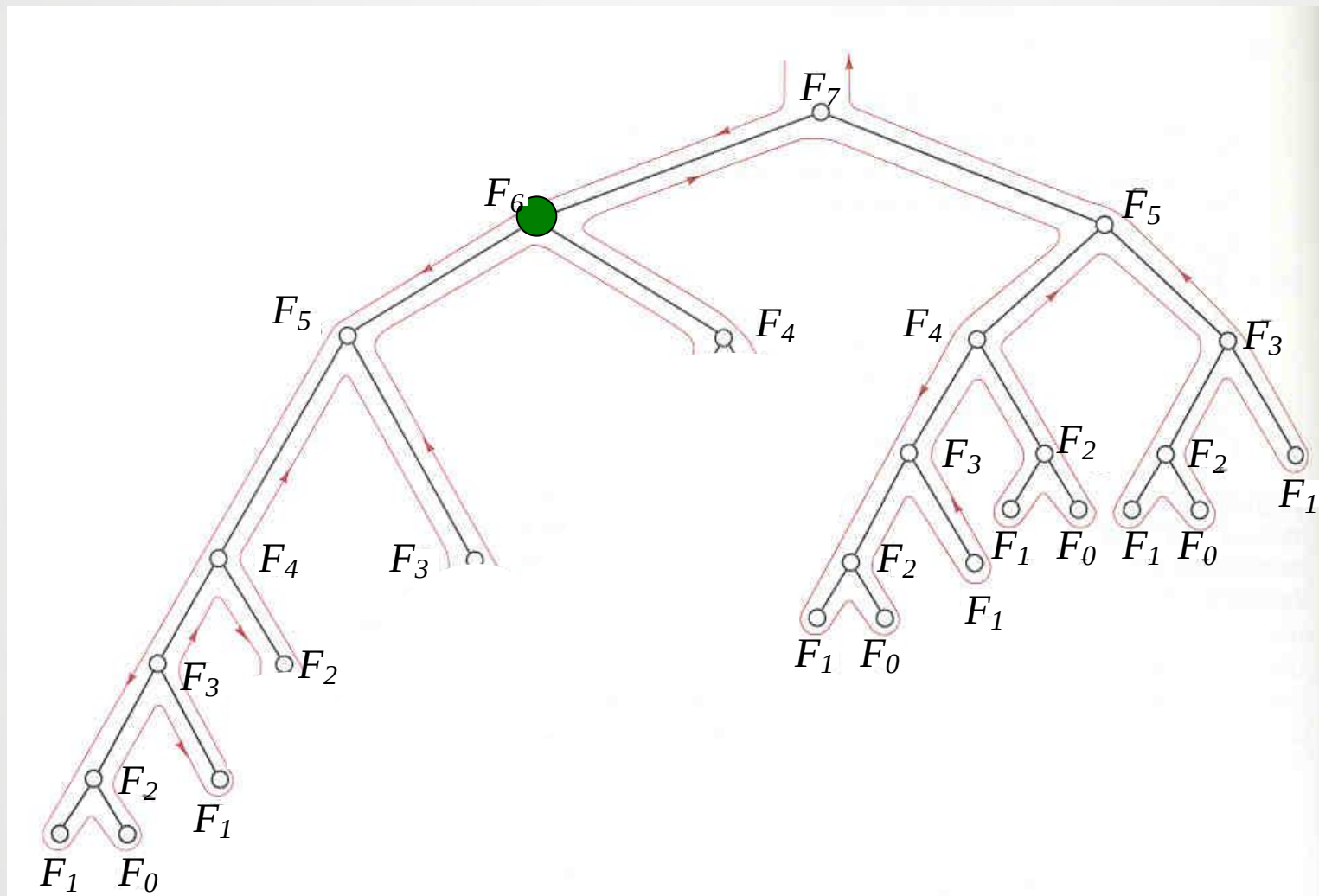
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	5
$v[5]$	8
$v[6]$	-1
$v[7]$	-1

再计算 $F(7)$



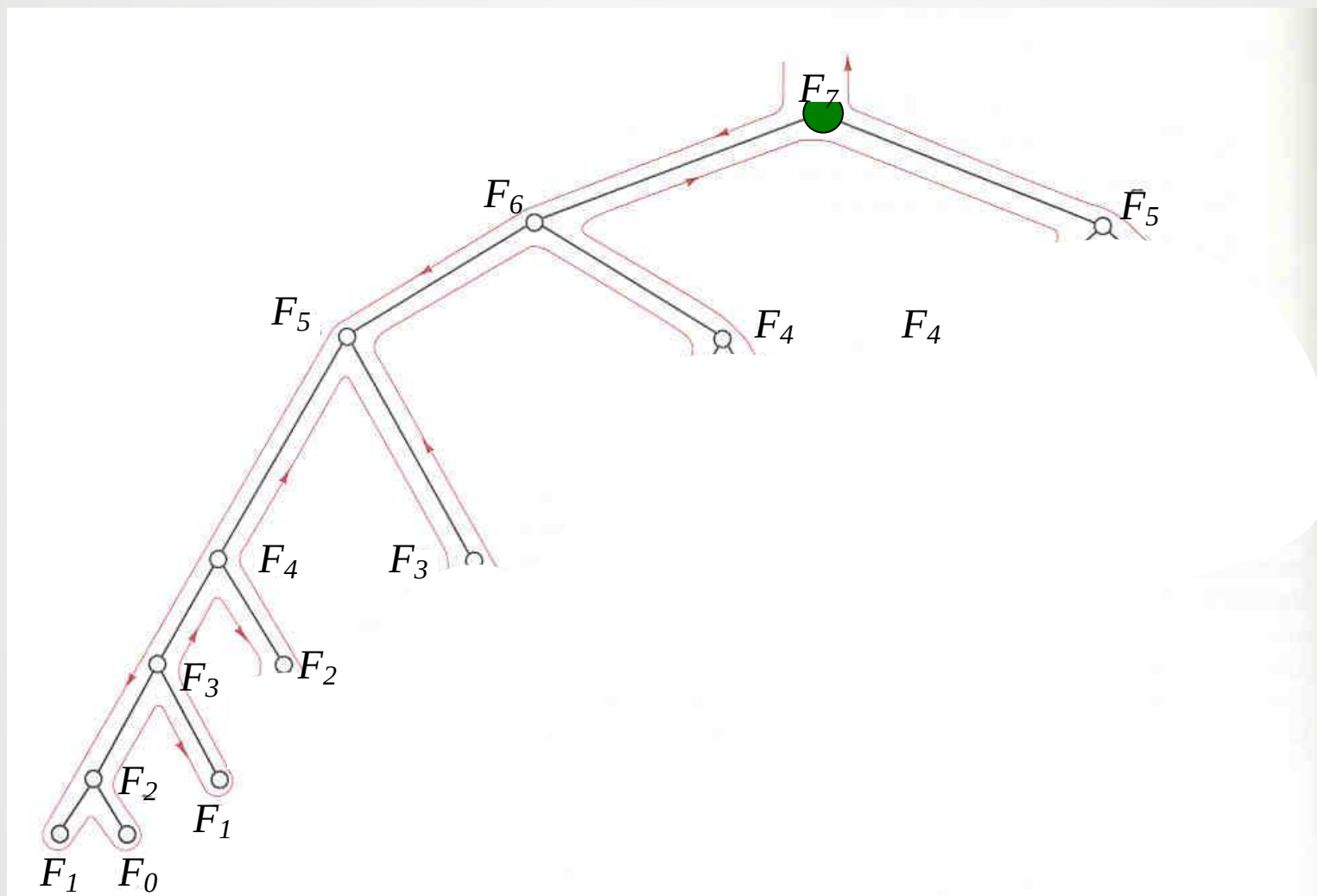
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	5
$v[5]$	8
$v[6]$	-1
$v[7]$	-1

再计算 $F(7)$



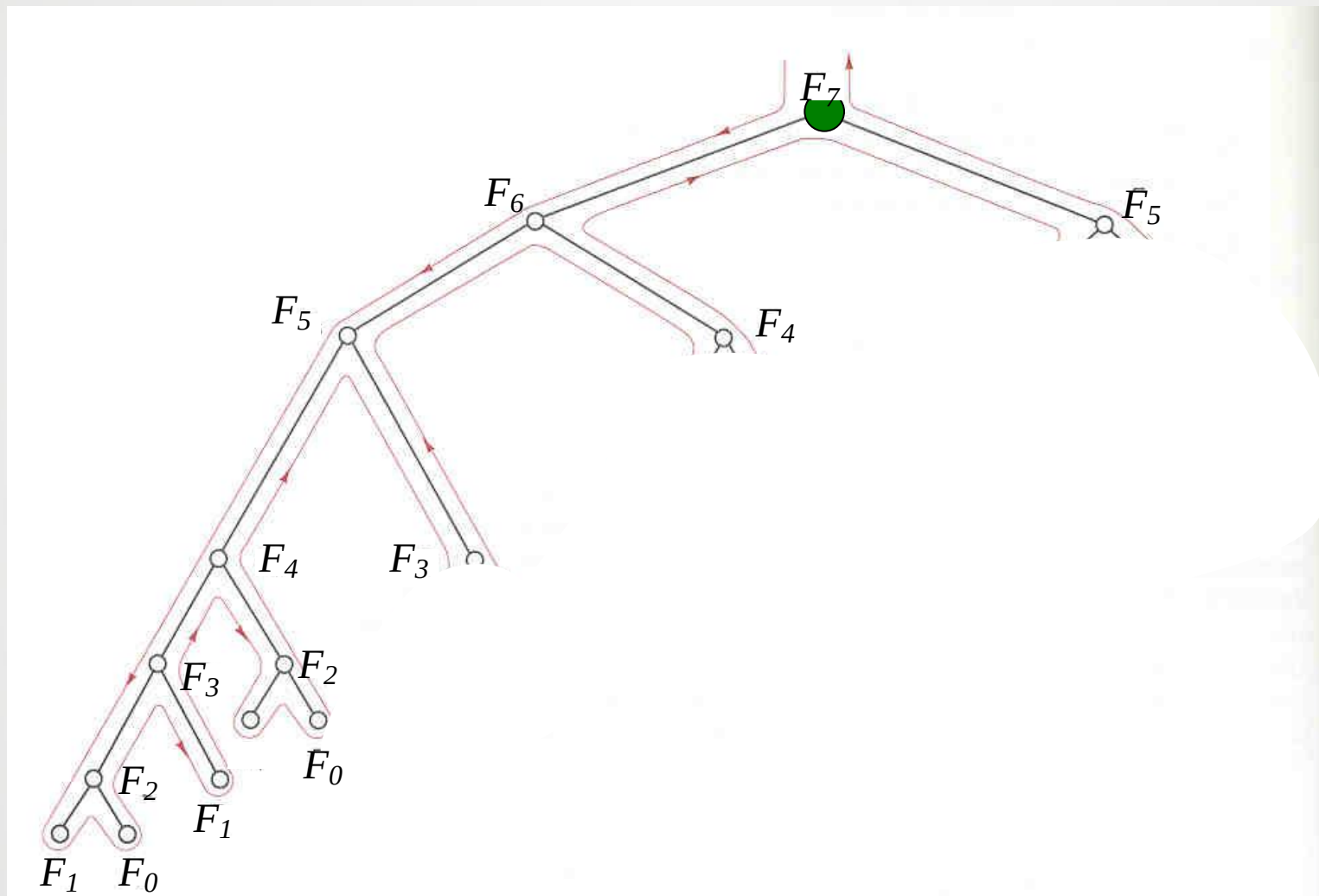
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	5
$v[5]$	8
$v[6]$	13
$v[7]$	-1

再计算 $F(7)$



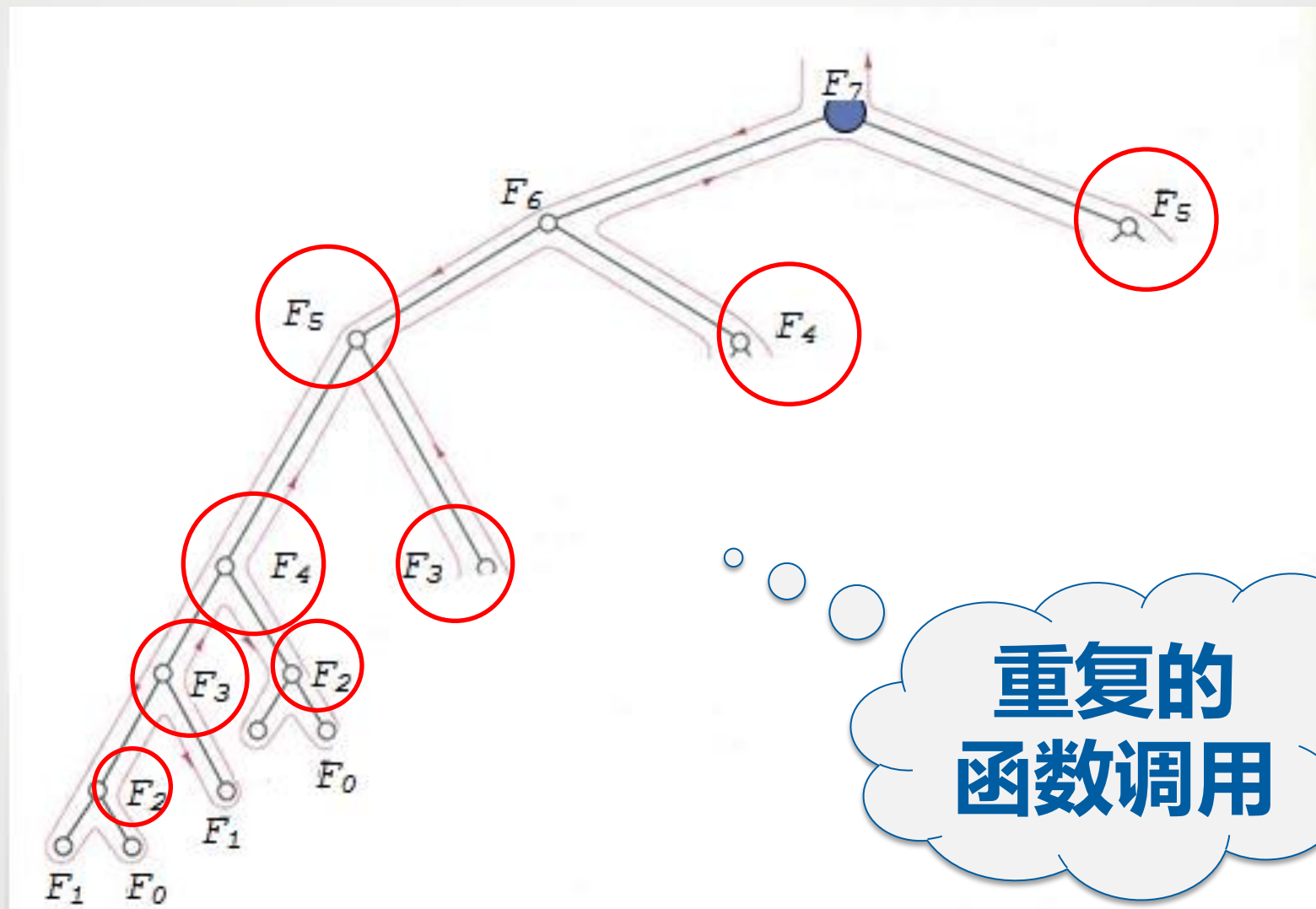
$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	5
$v[5]$	8
$v[6]$	13
$v[7]$	-1

再计算 $F(7)$



$v[0]$	1
$v[1]$	1
$v[2]$	2
$v[3]$	3
$v[4]$	5
$v[5]$	8
$v[6]$	13
$v[7]$	21

Can we do even better?

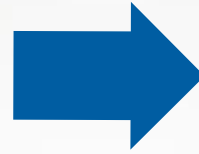


Can we do even better?

自顶向下

Top-down

- 递归



自底向上

Bottom-up

- 递推

递推方式
节约大量无用
递归调用

$F(n)$

```
1   A[0] = A[1] ← 1
2   for i ← 2 to n do
3       A[i] ← A[i-1] + A[i-2]
4   return A[n]
```

分治递归 vs 动态规划

分治递归:

$F(n)$

```
1  if  $n=0$  or  $n=1$  then
2      return 1
3  else
4      return  $F(n-1) + F(n-2)$ 
```

效率低!

动态规划:

$F(n)$

```
1   $A[0] = A[1] \leftarrow 1$ 
2  for  $i \leftarrow 2$  to  $n$  do
3       $A[i] \leftarrow A[i-1] + A[i-2]$ 
4  return  $A[n]$ 
```

高效!
时间复杂度 $O(n)$



方法的总结

- 写出一个递归公式，这个公式给出了问题与其子问题的解之间的关系
- E.g. $F(n) = F(n-1) + F(n-2)$.
- 用表(通常用数组)记录子问题的解,以便保存和以后的检索以从最简单问题的解填起，以自底向上的方式填表。
 - 这就保证了，当我们求解一个子问题的时候，所有与子相关的子子问题，都可以从表中直接取用，而不必重新计算。

由于20世纪40年代末期，还没有出现计算机，
这个动态填表的方法称为动态规划

谢谢大家